

Proximos Passos de Implementacao

Este guia descreve o proximo bloco recomendado de desenvolvimento do projeto `nexar-dashcam-collision-prediction`.

O foco agora e transformar os resultados atuais em um protocolo experimental mais defensavel:

1. Criar um split fixo por video.
2. Treinar usando esse split fixo.
3. Avaliar alertas somente no holdout de validacao.
4. Rodar o experimento com transfer learning.
5. Comparar os resultados antes de partir para CNN + GRU/LSTM.

O ponto mais importante e evitar que a avaliacao temporal use videos que tambem foram usados no treino. Hoje o treinamento separa 80/20 por video, mas a avaliacao temporal usa por padrao `data/interim/sample_100_videos.csv`, com os 100 videos. Isso pode inflar ou confundir as metricas.

Ordem recomendada

Execute nesta ordem:

1. Criar o arquivo `scripts/create_video_split.py`.
 2. Substituir `src/nexar_collision/models/train.py`.
 3. Substituir `scripts/train_baseline.py`.
 4. Substituir `src/nexar_collision/evaluation/evaluate.py`.
 5. Substituir `scripts/evaluate_model.py`.
 6. Substituir `scripts/sweep_alert_thresholds.py`.
 7. Gerar `data/interim/sample_100_videos_splits.csv`.
 8. Retreinar o modelo temporal com split fixo.
 9. Reavaliar somente no split `val`.
 10. Rodar a versao com `--pretrained`.
 11. Atualizar os relatorios.
-

Etapa 1 - Criar split fixo por video

Crie o arquivo:

```
scripts/create_video_split.py
```

Codigo completo:

```
from __future__ import annotations

import argparse
```

```
from pathlib import Path

import pandas as pd

PROJECT_ROOT = Path(__file__).resolve().parents[1]

def create_video_split(
    input_csv: Path,
    output_csv: Path,
    val_size: float,
    random_state: int,
    split_column: str,
) -> pd.DataFrame:
    sample_df = pd.read_csv(input_csv, dtype={"id": str})

    required_columns = {"id", "target"}
    missing_columns = required_columns.difference(sample_df.columns)
    if missing_columns:
        raise ValueError(
            f"Missing required columns in {input_csv}: {sorted(missing_columns)}"
        )

    video_df = sample_df[["id", "target"]].drop_duplicates("id").copy()

    duplicated_ids = video_df["id"].duplicated().sum()
    if duplicated_ids:
        raise ValueError(f"Found duplicated video ids: {duplicated_ids}")

    val_video_ids = (
        video_df.groupby("target", group_keys=False)
        .sample(frac=val_size, random_state=random_state)["id"]
        .tolist()
    )
    val_video_ids = set(val_video_ids)

    split_df = video_df.copy()
    split_df[split_column] = split_df["id"].apply(
        lambda video_id: "val" if video_id in val_video_ids else "train"
    )

    output_df = sample_df.merge(
        split_df[["id", split_column]],
        on="id",
        how="left",
        validate="many_to_one",
    )

    missing_split = output_df[split_column].isna().sum()
    if missing_split:
        raise ValueError(f"Some rows did not receive a split: {missing_split}")

    output_csv.parent.mkdir(parents=True, exist_ok=True)
```

```

output_df.to_csv(output_csv, index=False)
return output_df

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Create a deterministic train/val split by video id."
    )
    parser.add_argument(
        "--input",
        type=Path,
        default=PROJECT_ROOT / "data" / "interim" / "sample_100_videos.csv",
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=PROJECT_ROOT / "data" / "interim" /
"sample_100_videos_splits.csv",
    )
    parser.add_argument("--val-size", type=float, default=0.2)
    parser.add_argument("--random-state", type=int, default=42)
    parser.add_argument("--split-column", default="split")
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    split_df = create_video_split(
        input_csv=args.input,
        output_csv=args.output,
        val_size=args.val_size,
        random_state=args.random_state,
        split_column=args.split_column,
    )

    summary = (
        split_df[["id", "target", args.split_column]]
        .drop_duplicates("id")
        .groupby([args.split_column, "target"])
        .size()
        .rename("videos")
        .reset_index()
    )

    print(f"Saved split file to: {args.output}")
    print(summary.to_string(index=False))

if __name__ == "__main__":
    main()

```

Comando para gerar o split:

```
.\venv\Scripts\python.exe scripts\create_video_split.py --input
data\interim\sample_100_videos.csv --output
data\interim\sample_100_videos_splits.csv --val-size 0.2 --random-state 42
```

Resultado esperado:

split	target	videos
train	0	40
train	1	40
val	0	10
val	1	10

Etapa 2 - Substituir o treino para aceitar split fixo

Substitua o arquivo:

```
src/nexar_collision/models/train.py
```

Codigo completo:

```
"""Training loop utilities for the baseline frame classifier."""

from __future__ import annotations

import json
import random
from dataclasses import asdict, dataclass
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
from torch import nn
from torch.utils.data import DataLoader
from torchvision import transforms

from nexar_collision.data.dataset import FrameCollisionDataset
from nexar_collision.evaluation.metrics import compute_metrics
from nexar_collision.models.baseline_cnn import build_baseline_cnn
```

```

PROJECT_ROOT = Path(__file__).resolve().parents[3]

@dataclass
class TrainingConfig:
    manifest_path: Path = PROJECT_ROOT / "data" / "interim" /
    "sample_frames_manifest.csv"
    predictions_path: Path = PROJECT_ROOT / "outputs" / "predictions" /
    "baseline_frame_predictions.csv"
    metrics_path: Path = PROJECT_ROOT / "models" / "reports" /
    "baseline_metrics.json"
    checkpoint_path: Path = PROJECT_ROOT / "models" / "checkpoints" /
    "baseline_resnet18.pt"
    figures_dir: Path = PROJECT_ROOT / "outputs" / "figures"
    split_manifest_path: Path | None = None
    split_column: str = "split"
    train_split: str = "train"
    val_split: str = "val"
    figure_prefix: str = "baseline"
    batch_size: int = 16
    epochs: int = 3
    learning_rate: float = 1e-4
    val_size: float = 0.2
    random_state: int = 42
    num_workers: int = 0
    pretrained: bool = False
    device: str = "auto"

def resolve_device(device: str) -> torch.device:
    if device != "auto":
        return torch.device(device)
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def load_manifest_with_optional_split(config: TrainingConfig) -> pd.DataFrame:
    manifest_df = pd.read_csv(config.manifest_path, dtype={"id": str})

    if config.split_manifest_path is None:
        return manifest_df

    split_df = pd.read_csv(config.split_manifest_path, dtype={"id": str})
    required_columns = {"id", config.split_column}
    missing_columns = required_columns.difference(split_df.columns)
    if missing_columns:

```

```

        raise ValueError(
            f"Missing required columns in split manifest:
{sorted(missing_columns)}"
        )

    split_df = split_df[["id", config.split_column]].drop_duplicates("id")
    merged_df = manifest_df.merge(split_df, on="id", how="left",
validate="many_to_one")

    missing_split = merged_df[config.split_column].isna()
    if missing_split.any():
        missing_ids = merged_df.loc[missing_split,
"id"].drop_duplicates().head(10).tolist()
        raise ValueError(
            "Some manifest video ids were not found in the split manifest. "
            f"Examples: {missing_ids}"
        )

    return merged_df

def split_by_video(
    manifest_df: pd.DataFrame,
    val_size: float,
    random_state: int,
    split_column: str = "split",
    train_split: str = "train",
    val_split: str = "val",
) -> tuple[pd.DataFrame, pd.DataFrame]:
    if split_column in manifest_df.columns:
        train_df = manifest_df[manifest_df[split_column] == train_split].copy()
        val_df = manifest_df[manifest_df[split_column] == val_split].copy()

        if train_df.empty:
            raise ValueError(f"No rows found for train split: {train_split}")
        if val_df.empty:
            raise ValueError(f"No rows found for validation split: {val_split}")

        overlap = set(train_df["id"]).intersection(set(val_df["id"]))
        if overlap:
            raise ValueError(f"Found video leakage between train and val:
{sorted(overlap)[:10]}")

        return train_df, val_df

    stratify_col = "video_target" if "video_target" in manifest_df.columns else
"target"
    video_df = manifest_df[["id", stratify_col]].drop_duplicates("id")
    val_video_ids = (
        video_df.groupby(stratify_col, group_keys=False)
        .sample(frac=val_size, random_state=random_state)["id"]
        .tolist()
    )

```

```
val_df = manifest_df[manifest_df["id"].isin(val_video_ids)].copy()
train_df = manifest_df[~manifest_df["id"].isin(val_video_ids)].copy()
return train_df, val_df

def build_transforms() -> tuple[transforms.Compose, transforms.Compose]:
    train_transform = transforms.Compose(
        [
            transforms.Resize((224, 224)),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225],
            ),
        ]
    )
    eval_transform = transforms.Compose(
        [
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225],
            ),
        ]
    )
    return train_transform, eval_transform

def train_one_epoch(
    model: nn.Module,
    loader: DataLoader,
    optimizer: torch.optim.Optimizer,
    criterion: nn.Module,
    device: torch.device,
) -> float:
    model.train()
    losses = []

    for batch in loader:
        images = batch["image"].to(device)
        targets = batch["target"].to(device)

        optimizer.zero_grad(set_to_none=True)
        logits = model(images)
        loss = criterion(logits, targets)
        loss.backward()
        optimizer.step()
        losses.append(loss.item())

    return float(np.mean(losses))
```

```
@torch.no_grad()
def predict(
    model: nn.Module,
    loader: DataLoader,
    device: torch.device,
) -> pd.DataFrame:
    model.eval()
    records = []

    for batch in loader:
        images = batch["image"].to(device)
        logits = model(images)
        scores = torch.softmax(logits, dim=1)[ :, 1].cpu().numpy()
        preds = (scores >= 0.5).astype(int)

        for idx, score in enumerate(scores):
            records.append(
                {
                    "id": batch["video_id"][idx],
                    "frame_label": batch["frame_label"][idx],
                    "timestamp": float(batch["timestamp"][idx]),
                    "target": int(batch["target"][idx]),
                    "score": float(score),
                    "prediction": int(preds[idx]),
                }
            )

    return pd.DataFrame(records)

def aggregate_video_predictions(predictions_df: pd.DataFrame) -> pd.DataFrame:
    video_df = (
        predictions_df.groupby("id")
        .agg(
            target=("target", "first"),
            score_mean=("score", "mean"),
            score_max=("score", "max"),
        )
        .reset_index()
    )
    video_df["prediction_mean"] = (video_df["score_mean"] >= 0.5).astype(int)
    video_df["prediction_max"] = (video_df["score_max"] >= 0.5).astype(int)
    return video_df

def save_figures(
    predictions_df: pd.DataFrame,
    figures_dir: Path,
    figure_prefix: str,
) -> None:
    figures_dir.mkdir(parents=True, exist_ok=True)
    y_true = predictions_df["target"]
    y_pred = predictions_df["prediction"]
    y_score = predictions_df["score"]
```



```

ConfusionMatrixDisplay.from_predictions(y_true, y_pred)
plt.title(f"{figure_prefix} Frame Confusion Matrix")
plt.tight_layout()
plt.savefig(figures_dir / f"{figure_prefix}_frame_confusion_matrix.png",
dpi=150)
plt.close()

if len(np.unique(y_true)) == 2:
    RocCurveDisplay.from_predictions(y_true, y_score)
    plt.title(f"{figure_prefix} Frame ROC Curve")
    plt.tight_layout()
    plt.savefig(figures_dir / f"{figure_prefix}_frame_roc_curve.png", dpi=150)
    plt.close()

def build_class_weighted_loss(
    train_df: pd.DataFrame,
    device: torch.device,
) -> nn.Module:
    class_counts = train_df["target"].value_counts().sort_index()
    if len(class_counts) != 2:
        return nn.CrossEntropyLoss()

    total = float(class_counts.sum())
    class_weights = torch.tensor(
        [total / (2.0 * class_counts.get(index, 1)) for index in [0, 1]],
        dtype=torch.float32,
        device=device,
    )
    return nn.CrossEntropyLoss(weight=class_weights)

def train_model(config: TrainingConfig | None = None) -> dict[str, object]:
    config = config or TrainingConfig()
    seed_everything(config.random_state)
    device = resolve_device(config.device)

    manifest_df = load_manifest_with_optional_split(config)
    train_df, val_df = split_by_video(
        manifest_df=manifest_df,
        val_size=config.val_size,
        random_state=config.random_state,
        split_column=config.split_column,
        train_split=config.train_split,
        val_split=config.val_split,
    )
    train_transform, eval_transform = build_transforms()

    train_loader = DataLoader(
        FrameCollisionDataset(train_df, transform=train_transform),
        batch_size=config.batch_size,
        shuffle=True,
        num_workers=config.num_workers,

```

```

    )
    val_loader = DataLoader(
        FrameCollisionDataset(val_df, transform=eval_transform),
        batch_size=config.batch_size,
        shuffle=False,
        num_workers=config.num_workers,
    )

    model = build_baseline_cnn(pretrained=config.pretrained).to(device)
    criterion = build_class_weighted_loss(train_df, device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=config.learning_rate)

    history = []
    for epoch in range(1, config.epochs + 1):
        train_loss = train_one_epoch(model, train_loader, optimizer, criterion,
device)
        predictions_df = predict(model, val_loader, device)
        frame_metrics = compute_metrics(
            predictions_df["target"],
            predictions_df["prediction"],
            predictions_df["score"],
        )
        history.append({"epoch": epoch, "train_loss": train_loss,
**frame_metrics})
        print(f"epoch={epoch} train_loss={train_loss:.4f} val_f1=
{frame_metrics['f1']:.4f}")

        predictions_df = predict(model, val_loader, device)
        video_predictions_df = aggregate_video_predictions(predictions_df)

        frame_metrics = compute_metrics(
            predictions_df["target"],
            predictions_df["prediction"],
            predictions_df["score"],
        )
        video_metrics_mean = compute_metrics(
            video_predictions_df["target"],
            video_predictions_df["prediction_mean"],
            video_predictions_df["score_mean"],
        )
        video_metrics_max = compute_metrics(
            video_predictions_df["target"],
            video_predictions_df["prediction_max"],
            video_predictions_df["score_max"],
        )

        config.predictions_path.parent.mkdir(parents=True, exist_ok=True)
        config.metrics_path.parent.mkdir(parents=True, exist_ok=True)
        config.checkpoint_path.parent.mkdir(parents=True, exist_ok=True)

        predictions_df.to_csv(config.predictions_path, index=False)
        video_predictions_path = config.predictions_path.with_name(
            config.predictions_path.name.replace("_frame_predictions.csv",
"_video_predictions.csv"))

```

```

)
video_predictions_df.to_csv(video_predictions_path, index=False)
save_figures(predictions_df, config.figures_dir, config.figure_prefix)
torch.save(model.state_dict(), config.checkpoint_path)

report = {
    "config": {key: str(value) for key, value in asdict(config).items()},
    "device": str(device),
    "cuda_available": torch.cuda.is_available(),
    "train_frames": len(train_df),
    "val_frames": len(val_df),
    "train_videos": train_df["id"].nunique(),
    "val_videos": val_df["id"].nunique(),
    "train_video_ids": sorted(train_df["id"].drop_duplicates().tolist()),
    "val_video_ids": sorted(val_df["id"].drop_duplicates().tolist()),
    "train_target_distribution": {
        str(key): int(value)
        for key, value in
train_df["target"].value_counts().sort_index().items()
    },
    "val_target_distribution": {
        str(key): int(value)
        for key, value in val_df["target"].value_counts().sort_index().items()
    },
    "history": history,
    "frame_metrics": frame_metrics,
    "video_metrics_mean": video_metrics_mean,
    "video_metrics_max": video_metrics_max,
}

config.metrics_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
return report

```

Etapa 3 - Atualizar CLI de treino

Substitua o arquivo:

```
scripts/train_baseline.py
```

Codigo completo:

```

from __future__ import annotations

import argparse
import os
import sys
from pathlib import Path

```

```

PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"
os.environ.setdefault("MPLCONFIGDIR", str(PROJECT_ROOT / ".matplotlib"))
sys.path.insert(0, str(SRC_DIR))

from nexar_collision.models.train import TrainingConfig, train_model

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Train the ResNet18 frame
baseline.")
    parser.add_argument("--epochs", type=int, default=3)
    parser.add_argument("--batch-size", type=int, default=16)
    parser.add_argument("--learning-rate", type=float, default=1e-4)
    parser.add_argument("--device", default="auto")
    parser.add_argument("--pretrained", action="store_true")
    parser.add_argument("--num-workers", type=int, default=0)
    parser.add_argument(
        "--manifest",
        type=Path,
        default=PROJECT_ROOT / "data" / "interim" / "sample_frames_manifest.csv",
    )
    parser.add_argument(
        "--split-manifest",
        type=Path,
        default=None,
        help="Optional CSV with columns id and split.",
    )
    parser.add_argument("--split-column", default="split")
    parser.add_argument("--train-split", default="train")
    parser.add_argument("--val-split", default="val")
    parser.add_argument("--experiment-name", default="baseline")
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    experiment_name = args.experiment_name
    report = train_model(
        TrainingConfig(
            manifest_path=args.manifest,
            split_manifest_path=args.split_manifest,
            split_column=args.split_column,
            train_split=args.train_split,
            val_split=args.val_split,
            predictions_path=PROJECT_ROOT
            / "outputs"
            / "predictions"
            / f"{experiment_name}_frame_predictions.csv",
            metrics_path=PROJECT_ROOT
            / "models"
            / "reports"
            / f"{experiment_name}_metrics.json",

```

```

        checkpoint_path=PROJECT_ROOT
        / "models"
        / "checkpoints"
        / f"{experiment_name}_resnet18.pt",
        figure_prefix=experiment_name,
        epochs=args.epochs,
        batch_size=args.batch_size,
        learning_rate=args.learning_rate,
        device=args.device,
        pretrained=args.pretrained,
        num_workers=args.num_workers,
    )
)
print("Training complete")
print(f"Device: {report['device']}")
print(f"Train videos: {report['train_videos']}")
print(f"Validation videos: {report['val_videos']}")
print(f"Frame metrics: {report['frame_metrics']}")
print(f"Video metrics mean: {report['video_metrics_mean']}")
print(f"Video metrics max: {report['video_metrics_max']}")

if __name__ == "__main__":
    main()

```

Etapa 4 - Atualizar avaliacao temporal

Substitua o arquivo:

```
src/nexar_collision/evaluation/evaluate.py
```

Codigo completo:

```

"""Temporal alert evaluation for collision prediction models."""

from __future__ import annotations

import json
from dataclasses import asdict, dataclass
from pathlib import Path

import cv2
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

```

```

import torch
from PIL import Image

from nexar_collision.models.baseline_cnn import build_baseline_cnn
from nexar_collision.models.train import build_transforms, resolve_device

PROJECT_ROOT = Path(__file__).resolve().parents[3]

@dataclass
class AlertEvaluationConfig:
    sample_csv: Path = PROJECT_ROOT / "data" / "interim" / "sample_100_videos.csv"
    checkpoint_path: Path = PROJECT_ROOT / "models" / "checkpoints" /
    "baseline_resnet18.pt"
    risk_scores_path: Path = PROJECT_ROOT / "outputs" / "predictions" /
    "temporal_risk_scores.csv"
    alert_predictions_path: Path = PROJECT_ROOT / "outputs" / "predictions" /
    "alert_predictions.csv"
    metrics_path: Path = PROJECT_ROOT / "models" / "reports" /
    "alert_metrics.json"
    figures_dir: Path = PROJECT_ROOT / "outputs" / "figures"
    split: str | None = None
    split_column: str = "split"
    fps: float = 1.0
    threshold: float = 0.5
    batch_size: int = 32
    device: str = "auto"
    max_videos: int | None = None

def load_sample_df(config: AlertEvaluationConfig) -> pd.DataFrame:
    sample_df = pd.read_csv(config.sample_csv, dtype={"id": str})

    if config.split is not None:
        if config.split_column not in sample_df.columns:
            raise ValueError(
                f"Requested split={config.split}, but column "
                f"{config.split_column!r} was not found in {config.sample_csv}"
            )
        sample_df = sample_df[sample_df[config.split_column] ==
config.split].copy()

    if config.max_videos is not None:
        sample_df = sample_df.head(config.max_videos).copy()

    if sample_df.empty:
        raise ValueError("No videos available for alert evaluation.")

    return sample_df

def video_duration(video_path: Path) -> float:
    capture = cv2.VideoCapture(str(video_path))

```

```

    if not capture.isOpened():
        raise ValueError(f"Could not open video: {video_path}")

    fps = capture.get(cv2.CAP_PROP_FPS)
    frame_count = capture.get(cv2.CAP_PROP_FRAME_COUNT)
    capture.release()
    return float(frame_count / fps) if fps else 0.0

def read_frame_from_capture(
    capture: cv2.VideoCapture,
    video_path: Path,
    timestamp: float,
) -> Image.Image:
    capture.set(cv2.CAP_PROP_POS_MSEC, timestamp * 1000.0)
    ok, frame = capture.read()

    if not ok or frame is None:
        raise ValueError(f"Could not read frame at {timestamp:.3f}s from {video_path}")

    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    return Image.fromarray(frame)

def timestamps_for_video(duration: float, fps: float) -> list[float]:
    if duration <= 0:
        return [0.0]

    step = 1.0 / fps
    timestamps = np.arange(0.0, max(duration, step), step)
    return [float(min(timestamp, max(duration - 0.05, 0.0))) for timestamp in timestamps]

@torch.no_grad()
def score_video(
    model: torch.nn.Module,
    video_path: Path,
    transform,
    device: torch.device,
    fps: float,
    batch_size: int,
) -> list[dict[str, float]]:
    duration = video_duration(video_path)
    timestamps = timestamps_for_video(duration, fps)
    records: list[dict[str, float]] = []
    capture = cv2.VideoCapture(str(video_path))
    if not capture.isOpened():
        raise ValueError(f"Could not open video: {video_path}")

    try:
        for start in range(0, len(timestamps), batch_size):
            batch_timestamps = timestamps[start : start + batch_size]

```

```

        images = [
            transform(read_frame_from_capture(capture, video_path, timestamp))
            for timestamp in batch_timestamps
        ]
        batch = torch.stack(images).to(device)
        logits = model(batch)
        scores = torch.softmax(logits, dim=1)[: , 1].cpu().numpy()

        for timestamp, score in zip(batch_timestamps, scores):
            records.append(
                {
                    "timestamp": round(float(timestamp), 3),
                    "risk_score": float(score),
                    "duration": round(duration, 3),
                }
            )
    finally:
        capture.release()

    return records

def first_alert_time(video_scores: pd.DataFrame, threshold: float) -> float |
None:
    alerts = video_scores[video_scores["risk_score"] >= threshold]
    if alerts.empty:
        return None
    return float(alerts.sort_values("timestamp").iloc[0]["timestamp"])

def compute_alert_metrics(alert_df: pd.DataFrame) -> dict[str, float | int |
None]:
    positives = alert_df[alert_df["target"] == 1].copy()
    negatives = alert_df[alert_df["target"] == 0].copy()

    positive_count = len(positives)
    negative_count = len(negatives)
    predicted_alerts = alert_df["predicted_alert_time"].notna().sum()
    negative_alerts = negatives["predicted_alert_time"].notna().sum()

    true_positive_alerts = positives[
        positives["predicted_alert_time"].notna()
        & (positives["predicted_alert_time"] <= positives["time_of_event"])
    ].copy()
    missed_events = positive_count - len(true_positive_alerts)

    alert_precision_denominator = len(true_positive_alerts) + negative_alerts
    alert_precision = (
        len(true_positive_alerts) / alert_precision_denominator
        if alert_precision_denominator
        else None
    )
    alert_recall = len(true_positive_alerts) / positive_count if positive_count
    else None

```



```

false_alarm_rate = negative_alerts / negative_count if negative_count else
None
missed_event_rate = missed_events / positive_count if positive_count else None

lead_times = true_positive_alerts["predicted_lead_time"].dropna()
alert_errors = true_positive_alerts["alert_time_error"].dropna()

return {
    "videos": int(len(alert_df)),
    "positive_videos": int(positive_count),
    "negative_videos": int(negative_count),
    "predicted_alerts": int(predicted_alerts),
    "true_positive_alerts": int(len(true_positive_alerts)),
    "false_positive_alerts": int(negative_alerts),
    "missed_events": int(missed_events),
    "alert_precision": float(alert_precision) if alert_precision is not None
else None,
    "alert_recall": float(alert_recall) if alert_recall is not None else None,
    "false_alarm_rate": float(false_alarm_rate) if false_alarm_rate is not
None else None,
    "missed_event_rate": float(missed_event_rate) if missed_event_rate is not
None else None,
    "mean_predicted_lead_time": float(lead_times.mean()) if not
lead_times.empty else None,
    "median_predicted_lead_time": float(lead_times.median()) if not
lead_times.empty else None,
    "mean_alert_time_error": float(alert_errors.mean()) if not
alert_errors.empty else None,
    "median_alert_time_error": float(alert_errors.median()) if not
alert_errors.empty else None,
}

def plot_risk_curves(
    risk_scores_df: pd.DataFrame,
    sample_df: pd.DataFrame,
    figures_dir: Path,
    threshold: float,
    figure_prefix: str,
) -> None:
    figures_dir.mkdir(parents=True, exist_ok=True)
    examples = []
    for target in [1, 0]:
        candidate = sample_df[sample_df["target"] == target].head(1)
        if not candidate.empty:
            examples.append(candidate.iloc[0])

    for row in examples:
        video_id = row["id"]
        target = int(row["target"])
        video_scores = risk_scores_df[risk_scores_df["id"] == video_id]

        if video_scores.empty:
            continue

```

```

plt.figure(figsize=(9, 4))
plt.plot(video_scores["timestamp"], video_scores["risk_score"],
marker="o", linewidth=1)
plt.axhline(threshold, color="red", linestyle="--", label="alert
threshold")
if target == 1:
    plt.axvline(float(row["time_of_alert"]), color="orange", linestyle="--
", label="true alert")
    plt.axvline(float(row["time_of_event"]), color="black", linestyle="--
", label="event")
plt.ylim(0, 1)
plt.xlabel("Timestamp (s)")
plt.ylabel("Risk score")
plt.title(f"Temporal Risk Curve - video {video_id} target={target}")
plt.legend()
plt.tight_layout()
plt.savefig(figures_dir / f"
{figure_prefix}_temporal_risk_curve_{video_id}.png", dpi=150)
plt.close()

def evaluate_alerts(config: AlertEvaluationConfig | None = None) -> dict[str,
object]:
    config = config or AlertEvaluationConfig()
    device = resolve_device(config.device)
    _, eval_transform = build_transforms()

    sample_df = load_sample_df(config)

    model = build_baseline_cnn(pretrained=False).to(device)
    state_dict = torch.load(config.checkpoint_path, map_location=device)
    model.load_state_dict(state_dict)
    model.eval()

    risk_records: list[dict[str, object]] = []
    alert_records: list[dict[str, object]] = []

    total_videos = len(sample_df)
    for video_index, (_, row) in enumerate(sample_df.iterrows(), start=1):
        video_id = row["id"]
        target = int(row["target"])
        video_path = Path(row["video_path"])
        print(f"scoring_video={video_index}/{total_videos} id={video_id} target=
{target}")

        scores = score_video(
            model=model,
            video_path=video_path,
            transform=eval_transform,
            device=device,
            fps=config.fps,
            batch_size=config.batch_size,
        )

```

```
video_scores = pd.DataFrame(scores)
predicted_alert_time = first_alert_time(video_scores, config.threshold)

for record in scores:
    risk_records.append(
        {
            "id": video_id,
            "target": target,
            **record,
        }
    )

    time_of_alert = float(row["time_of_alert"]) if
pd.notna(row["time_of_alert"]) else None
    time_of_event = float(row["time_of_event"]) if
pd.notna(row["time_of_event"]) else None
    predicted_lead_time = (
        time_of_event - predicted_alert_time
        if target == 1 and predicted_alert_time is not None and time_of_event
is not None
        else None
    )
    alert_time_error = (
        predicted_alert_time - time_of_alert
        if target == 1 and predicted_alert_time is not None and time_of_alert
is not None
        else None
    )

    alert_records.append(
        {
            "id": video_id,
            "target": target,
            "time_of_alert": time_of_alert,
            "time_of_event": time_of_event,
            "predicted_alert_time": predicted_alert_time,
            "predicted_lead_time": predicted_lead_time,
            "alert_time_error": alert_time_error,
            "max_risk_score": float(video_scores["risk_score"].max()),
            "mean_risk_score": float(video_scores["risk_score"].mean()),
        }
    )

risk_scores_df = pd.DataFrame(risk_records)
alert_df = pd.DataFrame(alert_records)
metrics = compute_alert_metrics(alert_df)

config.risk_scores_path.parent.mkdir(parents=True, exist_ok=True)
config.alert_predictions_path.parent.mkdir(parents=True, exist_ok=True)
config.metrics_path.parent.mkdir(parents=True, exist_ok=True)

risk_scores_df.to_csv(config.risk_scores_path, index=False)
alert_df.to_csv(config.alert_predictions_path, index=False)
figure_prefix = config.metrics_path.stem.replace("_alert_metrics", "")
```

```

    plot_risk_curves(
        risk_scores_df=risk_scores_df,
        sample_df=sample_df,
        figures_dir=config.figures_dir,
        threshold=config.threshold,
        figure_prefix=figure_prefix,
    )

    report = {
        "config": {key: str(value) for key, value in asdict(config).items()},
        "device": str(device),
        "cuda_available": torch.cuda.is_available(),
        "metrics": metrics,
    }
    config.metrics_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
    return report

def evaluate_model() -> dict[str, object]:
    return evaluate_alerts()

```

Etapa 5 - Atualizar CLI de avaliacao

Substitua o arquivo:

```
scripts/evaluate_model.py
```

Codigo completo:

```

from __future__ import annotations

import argparse
import os
import sys
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"
os.environ.setdefault("MPLCONFIGDIR", str(PROJECT_ROOT / ".matplotlib"))
sys.path.insert(0, str(SRC_DIR))

from nexar_collision.evaluation.evaluate import AlertEvaluationConfig,
evaluate_alerts

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(

```

```

        description="Evaluate temporal alert prediction from frame risk scores."
    )
    parser.add_argument("--fps", type=float, default=1.0)
    parser.add_argument("--threshold", type=float, default=0.5)
    parser.add_argument("--batch-size", type=int, default=32)
    parser.add_argument("--device", default="auto")
    parser.add_argument("--max-videos", type=int, default=None)
    parser.add_argument(
        "--sample-csv",
        type=Path,
        default=PROJECT_ROOT / "data" / "interim" / "sample_100_videos.csv",
    )
    parser.add_argument(
        "--checkpoint",
        type=Path,
        default=PROJECT_ROOT / "models" / "checkpoints" / "baseline_resnet18.pt",
    )
    parser.add_argument("--split", default=None)
    parser.add_argument("--split-column", default="split")
    parser.add_argument("--experiment-name", default="baseline")
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    report = evaluate_alerts(
        AlertEvaluationConfig(
            sample_csv=args.sample_csv,
            checkpoint_path=args.checkpoint,
            risk_scores_path=PROJECT_ROOT
            / "outputs"
            / "predictions"
            / f"{args.experiment_name}_temporal_risk_scores.csv",
            alert_predictions_path=PROJECT_ROOT
            / "outputs"
            / "predictions"
            / f"{args.experiment_name}_alert_predictions.csv",
            metrics_path=PROJECT_ROOT
            / "models"
            / "reports"
            / f"{args.experiment_name}_alert_metrics.json",
            split=args.split,
            split_column=args.split_column,
            fps=args.fps,
            threshold=args.threshold,
            batch_size=args.batch_size,
            device=args.device,
            max_videos=args.max_videos,
        )
    )

    print("Temporal alert evaluation complete")
    print(f"Device: {report['device']}")
    print(f"Metrics: {report['metrics']}")

```

```
if __name__ == "__main__":  
    main()
```

Etapa 6 - Atualizar sweep de thresholds

Substitua o arquivo:

```
scripts/sweep_alert_thresholds.py
```

Código completo:

```
from __future__ import annotations  
  
import argparse  
import os  
import sys  
from pathlib import Path  
  
PROJECT_ROOT = Path(__file__).resolve().parents[1]  
SRC_DIR = PROJECT_ROOT / "src"  
os.environ.setdefault("MPLCONFIGDIR", str(PROJECT_ROOT / ".matplotlib"))  
sys.path.insert(0, str(SRC_DIR))  
  
import matplotlib  
  
matplotlib.use("Agg")  
import matplotlib.pyplot as plt  
import numpy as np  
import pandas as pd  
  
from nexar_collision.evaluation.evaluate import compute_alert_metrics  
  
def load_sample_df(  
    sample_csv: Path,  
    split: str | None,  
    split_column: str,  
) -> pd.DataFrame:  
    sample_df = pd.read_csv(sample_csv, dtype={"id": str})  
  
    if split is not None:  
        if split_column not in sample_df.columns:  
            raise ValueError(  
                f"Requested split={split}, but column {split_column!r} "  
                f"was not found in {sample_csv}"  
            )
```

```

        sample_df = sample_df[sample_df[split_column] == split].copy()

    if sample_df.empty:
        raise ValueError("No videos available for threshold sweep.")

    return sample_df

def build_alert_predictions(
    risk_scores_df: pd.DataFrame,
    sample_df: pd.DataFrame,
    threshold: float,
) -> pd.DataFrame:
    records = []
    for _, row in sample_df.iterrows():
        video_id = row["id"]
        target = int(row["target"])
        video_scores = risk_scores_df[risk_scores_df["id"] == video_id]

        if video_scores.empty:
            predicted_alert_time = None
            max_risk_score = None
            mean_risk_score = None
        else:
            alerts = video_scores[video_scores["risk_score"] >= threshold]
            predicted_alert_time = (
                float(alerts.sort_values("timestamp").iloc[0]["timestamp"])
                if not alerts.empty
                else None
            )
            max_risk_score = float(video_scores["risk_score"].max())
            mean_risk_score = float(video_scores["risk_score"].mean())

            time_of_alert = float(row["time_of_alert"]) if
pd.notna(row["time_of_alert"]) else None
            time_of_event = float(row["time_of_event"]) if
pd.notna(row["time_of_event"]) else None
            predicted_lead_time = (
                time_of_event - predicted_alert_time
                if target == 1 and predicted_alert_time is not None and time_of_event
is not None
                else None
            )
            alert_time_error = (
                predicted_alert_time - time_of_alert
                if target == 1 and predicted_alert_time is not None and time_of_alert
is not None
                else None
            )

            records.append(
                {
                    "id": video_id,
                    "target": target,

```

```

        "time_of_alert": time_of_alert,
        "time_of_event": time_of_event,
        "predicted_alert_time": predicted_alert_time,
        "predicted_lead_time": predicted_lead_time,
        "alert_time_error": alert_time_error,
        "max_risk_score": max_risk_score,
        "mean_risk_score": mean_risk_score,
    }
)

return pd.DataFrame(records)

def plot_threshold_sweep(sweep_df: pd.DataFrame, output_path: Path) -> None:
    output_path.parent.mkdir(parents=True, exist_ok=True)

    plt.figure(figsize=(9, 5))
    plt.plot(sweep_df["threshold"], sweep_df["alert_recall"], marker="o",
label="alert recall")
    plt.plot(sweep_df["threshold"], sweep_df["alert_precision"], marker="o",
label="alert precision")
    plt.plot(sweep_df["threshold"], sweep_df["false_alarm_rate"], marker="o",
label="false alarm rate")
    plt.xlabel("Alert threshold")
    plt.ylabel("Metric")
    plt.ylim(0, 1)
    plt.title("Temporal Alert Threshold Sweep")
    plt.legend()
    plt.tight_layout()
    plt.savefig(output_path, dpi=150)
    plt.close()

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Sweep temporal alert
thresholds.")
    parser.add_argument(
        "--risk-scores",
        type=Path,
        default=None,
    )
    parser.add_argument(
        "--sample-csv",
        type=Path,
        default=PROJECT_ROOT / "data" / "interim" / "sample_100_videos.csv",
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=None,
    )
    parser.add_argument("--experiment-name", default="baseline")
    parser.add_argument("--split", default=None)
    parser.add_argument("--split-column", default="split")

```



```
parser.add_argument("--start", type=float, default=0.1)
parser.add_argument("--stop", type=float, default=0.99)
parser.add_argument("--step", type=float, default=0.01)
return parser.parse_args()

def main() -> None:
    args = parse_args()
    risk_scores_path = args.risk_scores or (
        PROJECT_ROOT
        / "outputs"
        / "predictions"
        / f"{args.experiment_name}_temporal_risk_scores.csv"
    )
    output_path = args.output or (
        PROJECT_ROOT / "models" / "reports" / f"
{args.experiment_name}_alert_threshold_sweep.csv"
    )

    risk_scores_df = pd.read_csv(risk_scores_path, dtype={"id": str})
    sample_df = load_sample_df(args.sample_csv, args.split, args.split_column)

    records = []
    thresholds = np.round(np.arange(args.start, args.stop + args.step / 2,
args.step), 3)
    for threshold in thresholds:
        alert_df = build_alert_predictions(risk_scores_df, sample_df,
float(threshold))
        metrics = compute_alert_metrics(alert_df)
        records.append({"threshold": float(threshold), **metrics})

    sweep_df = pd.DataFrame(records)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    sweep_df.to_csv(output_path, index=False)
    plot_threshold_sweep(
        sweep_df,
        PROJECT_ROOT / "outputs" / "figures" / f"
{args.experiment_name}_alert_threshold_sweep.png",
    )

    candidate = sweep_df.sort_values(
        ["false_alarm_rate", "missed_event_rate", "mean_alert_time_error"],
        ascending=[True, True, False],
    ).head(1)
    print(f"Saved threshold sweep to: {output_path}")
    print("Candidate operating point:")
    print(candidate.to_string(index=False))

if __name__ == "__main__":
    main()
```

Etapa 7 - Rodar baseline temporal com protocolo corrigido

Depois de aplicar os codigos acima, gere o split:

```
.\env\Scripts\python.exe scripts\create_video_split.py --input
data\interim\sample_100_videos.csv --output
data\interim\sample_100_videos_splits.csv --val-size 0.2 --random-state 42
```

Retreine o modelo temporal atual, mas agora usando o split fixo:

```
.\env\Scripts\python.exe scripts\train_baseline.py --manifest
data\interim\temporal_frames_224_manifest.csv --split-manifest
data\interim\sample_100_videos_splits.csv --experiment-name
temporal_alert_224_split --epochs 3 --batch-size 64 --learning-rate 0.0001 --num-
workers 2
```

Avalie somente no holdout:

```
.\env\Scripts\python.exe scripts\evaluate_model.py --checkpoint
models\checkpoints\temporal_alert_224_split_resnet18.pt --experiment-name
temporal_alert_224_split --sample-csv data\interim\sample_100_videos_splits.csv --
split val --fps 1 --threshold 0.5 --batch-size 32
```

Rode o sweep tambem somente no holdout:

```
.\env\Scripts\python.exe scripts\sweep_alert_thresholds.py --experiment-name
temporal_alert_224_split --sample-csv data\interim\sample_100_videos_splits.csv --
split val
```

Arquivos esperados:

```
data/interim/sample_100_videos_splits.csv
models/checkpoints/temporal_alert_224_split_resnet18.pt
models/reports/temporal_alert_224_split_metrics.json
models/reports/temporal_alert_224_split_alert_metrics.json
models/reports/temporal_alert_224_split_alert_threshold_sweep.csv
outputs/predictions/temporal_alert_224_split_frame_predictions.csv
outputs/predictions/temporal_alert_224_split_temporal_risk_scores.csv
outputs/predictions/temporal_alert_224_split_alert_predictions.csv
outputs/figures/temporal_alert_224_split_frame_confusion_matrix.png
outputs/figures/temporal_alert_224_split_frame_roc_curve.png
outputs/figures/temporal_alert_224_split_alert_threshold_sweep.png
```

Etapa 8 - Rodar transfer learning

O código atual já suporta `--pretrained`. Depois que a validação corrigida estiver funcionando, rode:

```
.\env\Scripts\python.exe scripts\train_baseline.py --manifest
data\interim\temporal_frames_224_manifest.csv --split-manifest
data\interim\sample_100_videos_splits.csv --experiment-name
temporal_alert_224_pretrained --pretrained --epochs 8 --batch-size 64 --learning-
rate 0.00005 --num-workers 2
```

Observação: a primeira execução com `--pretrained` pode baixar pesos do ImageNet via `torchvision`. Se o download falhar por falta de internet, rode novamente em um ambiente com acesso ou remova temporariamente `--pretrained`.

Avalie:

```
.\env\Scripts\python.exe scripts\evaluate_model.py --checkpoint
models\checkpoints\temporal_alert_224_pretrained_resnet18.pt --experiment-name
temporal_alert_224_pretrained --sample-csv
data\interim\sample_100_videos_splits.csv --split val --fps 1 --threshold 0.5 --
batch-size 32
```

Rode o sweep:

```
.\env\Scripts\python.exe scripts\sweep_alert_thresholds.py --experiment-name
temporal_alert_224_pretrained --sample-csv
data\interim\sample_100_videos_splits.csv --split val
```

Etapa 9 - Como decidir se melhorou

Compare estes arquivos:

```
models/reports/temporal_alert_224_split_alert_metrics.json
models/reports/temporal_alert_224_pretrained_alert_metrics.json
models/reports/temporal_alert_224_split_alert_threshold_sweep.csv
models/reports/temporal_alert_224_pretrained_alert_threshold_sweep.csv
```

Indicadores principais:

alert_precision	deve subir
false_alarm_rate	deve cair
alert_recall	deve permanecer aceitável

```
missed_event_rate    nao deve explodir
mean_alert_time_error deve ficar menos negativo
```

Uma melhoria boa seria algo nesta direcao:

```
menos falsos alarmes
precision maior
recall ainda acima de 0.70 ou 0.80
alertas menos absurdamente cedo
```

Nao escolha o melhor threshold olhando so para **precision**. Um threshold muito alto pode zerar falsos alarmes, mas perder muitos eventos.

Um criterio pratico:

1. Filtrar thresholds com `alert_recall` ≥ 0.70 .
2. Entre eles, escolher o menor `false_alarm_rate`.
3. Em caso de empate, escolher maior `alert_precision`.
4. Em caso de novo empate, escolher `mean_alert_time_error` mais proximo de zero.

Etapa 10 - Atualizar relatorios

Depois de rodar os experimentos, atualize:

```
reports/temporal_label_progression.md
reports/methodology.md
reports/alert_evaluation.md
```

Inclua uma tabela assim:

Model	Eval split	Threshold	Precision	Recall	False alarm rate	Missed event rate	Mean lead time	Mean alert error
Temporal label ResNet18	val only	0.50	TBD	TBD	TBD	TBD	TBD	TBD
Temporal label ResNet18 pretrained	val only	0.50	TBD	TBD	TBD	TBD	TBD	TBD
Temporal label ResNet18 pretrained	val only	best sweep	TBD	TBD	TBD	TBD	TBD	TBD

Tambem registre:

- O split agora e fixo por video.
- A avaliacao temporal usa somente videos de validacao.
- As metricas antigas em 100 videos ficam como exploratorias, nao conclusivas.
- A proxima etapa cientifica apos isso e modelo temporal de janelas curtas.

Etapa 11 - Quando partir para CNN + GRU/LSTM

So avance para CNN + GRU/LSTM depois de ter:

1. split fixo salvo
2. treino reproduzivel
3. avaliacao temporal no holdout
4. baseline com e sem pretrained
5. threshold sweep no holdout
6. relatorio atualizado

O motivo: um modelo temporal mais complexo so sera util se a comparacao contra o baseline estiver justa. Caso contrario, fica dificil saber se a melhoria veio da arquitetura ou de vazamento/metrica inconsistente.

Checklist final

Marque quando concluir:

- [] scripts/create_video_split.py criado
- [] src/nexar_collision/models/train.py atualizado
- [] scripts/train_baseline.py atualizado
- [] src/nexar_collision/evaluation/evaluate.py atualizado
- [] scripts/evaluate_model.py atualizado
- [] scripts/sweep_alert_thresholds.py atualizado
- [] data/interim/sample_100_videos_splits.csv gerado
- [] temporal_alert_224_split treinado
- [] temporal_alert_224_split avaliado em val
- [] sweep de temporal_alert_224_split gerado em val
- [] temporal_alert_224_pretrained treinado
- [] temporal_alert_224_pretrained avaliado em val
- [] sweep de temporal_alert_224_pretrained gerado em val
- [] reports atualizados